Sketch of the Proof

① **Lyapunov function.** Define

$$V(x,y,z) = rx^2 + \sigma y^2 + \sigma(z-2r)^2.$$

Along solutions of the Lorenz system,

$$\dot V = -2\sigma\left(rx^2 + y^2 + bz^2 - 2brz\right).$$

② **Negativity outside a large ellipsoid.**

The quadratic form $Q(x,y,z) = rx^2 + y^2 + bz^2 - 2brz$ is positive definite away from a bounded region. Therefore

$$\dot V < 0 \quad \text{outside a sufficiently large ellipsoid.}$$

③ **Construct the sets D and E.**

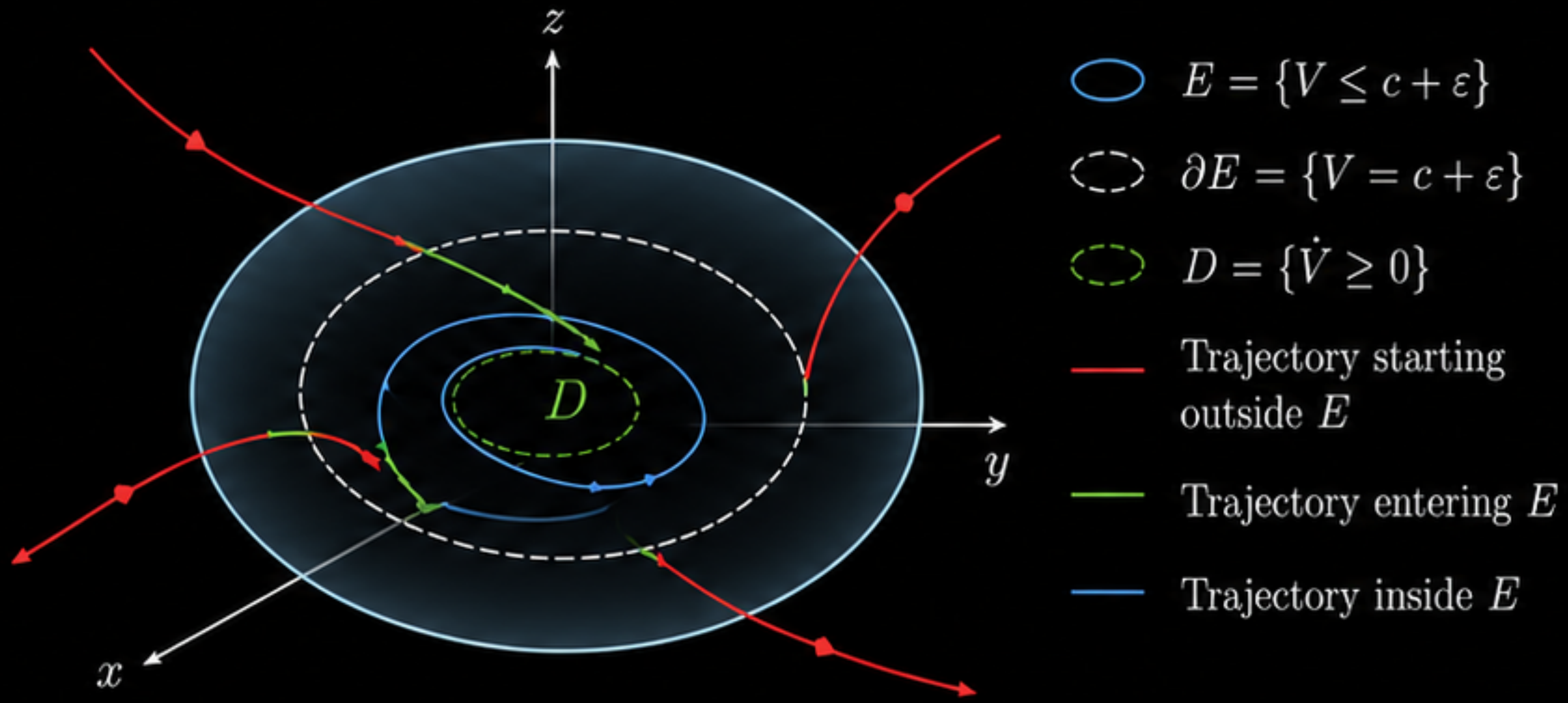
Let $D = \{\dot V \geq 0\}$. Then D is bounded (hence compact).

Let $c = \max_D V$ and $E = \{V \leq c + \varepsilon\}$

for some $\varepsilon > 0$. Then E is a bounded ellipsoid containing D.

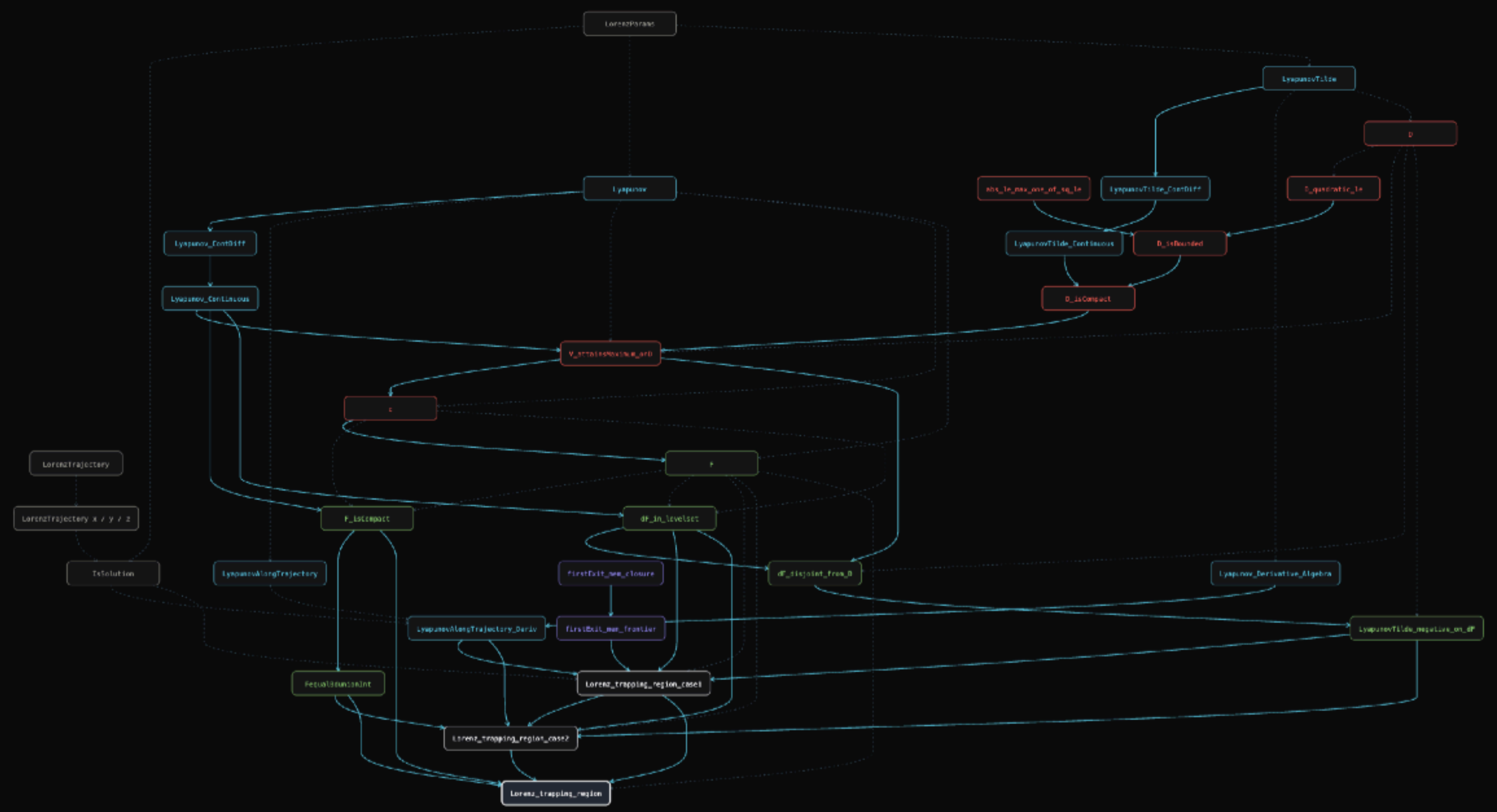
④ **Properties of E.**

- Outside E: $\dot V < 0$. Hence $V(t)$ strictly decreases along any trajectory that starts outside E.
- Thus every trajectory enters E in finite time.
- On the boundary $\partial E = \{V = c + \varepsilon\}$ we have $\dot V < 0$, so the vector field points strictly inward. Therefore trajectories cannot leave E once inside.



$\Rightarrow$ Therefore E is a trapping region for the Lorenz flow. Equivalently, $T(E) \subset \operatorname{int}(E)$.

- Phase 1 · The dynamical system
- Phase 2 · Defining V and Ṽ
- Phase 3 · The set D and sup c
- Phase 4 · The set E and ∂E
- Toolkit · replaces Case 1 subcases 1–2
- Phase 5 · Trapping region



STEP 31 OF 31 · THEOREM · **PHASE 5 · THEOREM**

Lorenz_trapping_region

"If $\phi(t_0) \in E$, then $\phi(t) \in \mathrm{int}(E)$ for all $t > t_0$." Four moves: **use F p** to supply the witness, **F_isCompact.isBounded** to discharge the boundedness conjunct, **FequalBdunionInt** to split F into interior ∪ frontier, and then each piece to its case. Note that the conclusion is **stronger than the hypothesis**: land anywhere in F — even exactly on the boundary — and at every later instant you are strictly inside. The trapping region does not merely hold you; it pulls you off its own edge.

```
theorem Lorenz_trapping_region (p) (φ)
    (hφ : IsSolution p φ) :
  ∃ F : Set (ℝ × ℝ × ℝ),
    Bornology.IsBounded F ∧
    ∀ t0 : ℝ, φ.total t0 ∈ F →
      (∀ t > t0, φ.total t ∈ interior F) := by
  use F p
  rw [and_iff_left ?_]
  · refine IsCompact.isBounded ?_
    apply F_isCompact
  · intro t0 h1
```